Checking encryption on the MT6991 image

Four read-only checks that tell you whether `/data` is encrypted, and whether the UFS controller's AES engine or the CPU does the work. Explained from scratch.

Written 1 November 2026 · MediaTek MT6991 SoC evaluation board (`k6991v1_64`) · Android GKI kernel 6.6 (`android15-6.6`) with MediaTek vendor modules · Companion to *UFS on MT6991: the bug, the fix, and the G5 puzzle*. It expands that document's section 4.5.

THE ANSWER ON ONE PAGE

1. There are two separate questions. **Is `/data` encrypted at all?** And if it is, **which piece of silicon does the AES maths**: the engine inside the UFS controller, or the CPU?
2. Only the engine case lets the encryption clock, `ufs_fde` (416 or 546 MHz), affect the G5 numbers. That's why the second question matters.
3. Four checks answer both in about two minutes. All of them only read: two properties, the `/data` mount line, one sysfs folder, and one line in the kernel log.
4. The line that settles it is `fscrypt: AES-256-XTS using blk-crypto (native)`. If you see it, the controller's engine encrypts every file block written to `/data`.
5. Found while writing this: if the CPU does the encryption inside the filesystem, f2fs silently turns fio's `direct=1` into ordinary buffered I/O for encrypted files. That alone could make numbers wander (section 3).
6. Nothing has been run on the board for this document yet. What the checks print will be the first real data on this question.

How to read this

Same conventions as the main write-up. Important claims carry one of three labels:

SEEN ON THE BOARD — Measured on the board: a screenshot, a dmesg line, a benchmark.

FROM THE CODE — I read it in the source copies in `sources/`, with the file and line.

NOT YET PROVEN — A hypothesis. The test that would settle it is always next to it.

Line numbers point into the copies in `sources/`: `android15-6.6/` (Android common kernel), `mtk_vendor_6.6/` (MediaTek's code from OnePlus's public MT6991 tree) and, new here, `upstream_v6.18/`: eight upstream Linux 6.18 files, because the fscrypt, f2fs and block-layer files of `android15-6.6` aren't on this machine.

Contents

1. The picture: where encryption can happen 2
2. The four checks 4
3. What each answer means for G5 8
4. Reference: changes, sources, glossary 10

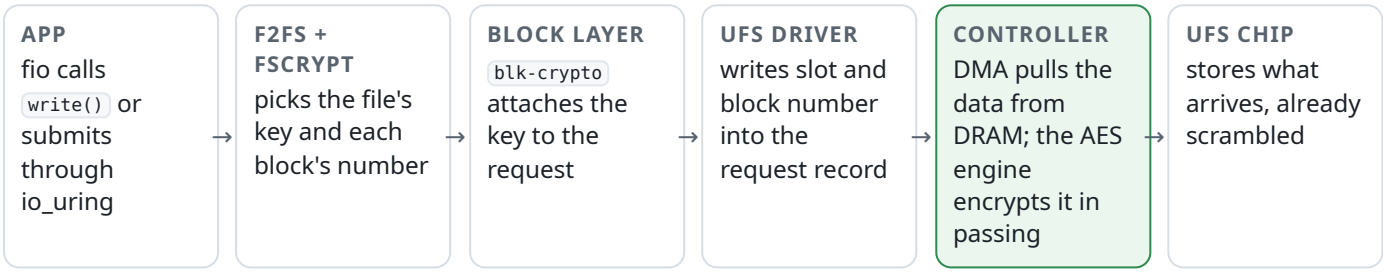
1 The picture: where encryption can happen

PLAIN WORDS

Encrypting `/data` means this: before a block of a file leaves for the flash chip, something scrambles it with AES and a secret key, and on the way back the same thing unscrambles it. The flash chip only ever stores the scrambled version, so someone who desolders it gets noise. Whether this happens at all, and which piece of silicon does the scrambling, depends on how the image was built.

1.1 The trip a write takes

When fio writes to a file on `/data`, the data passes down through the software and then out through the hardware. On the hardware path it looks like this:



This is path C in the table below. On the other two paths the CPU does the AES step earlier, and the controller passes the data through untouched.

1.2 Three places the AES maths can happen

PATH	WHERE AES RUNS	WHEN IT APPLIES	WHAT IT COSTS
A · in the filesystem	CPU, inside fscrypt, before the block layer	`/data` mounted *without* `inlinecrypt`	The CPU encrypts every byte into a spare "bounce" page and writes that. f2fs also refuses direct I/O on encrypted files (section 3).
B · in the block layer	CPU, inside `blk-crypto-fallback`	`inlinecrypt` set, but no engine registered	The CPU encrypts every byte, again through bounce pages. Direct I/O still works.
C · inline, in hardware	The AES engine inside the UFS controller	`inlinecrypt` set, and the engine registered	No AES work on the CPU. The engine's speed is set by its clock, `ufs_fde`.

KEY POINT

Two switches pick the path. The mount option `inlinecrypt` decides between A and the other two. Whether the controller registered its engine decides between B and C. Checks 2 and 3 read those two switches, and check 4 reads the kernel's own note of which path it took.

1.3 How the engine works

The engine is a block of logic inside the UFS host controller, the part of the MT6991 that talks to the storage chip. Its design comes from the UFS host controller standard (UFSHCI), so every UFS controller with crypto works the same way:

1. **Key slots.** The engine has a small set of registers called key slots, each holding one AES key. When a file's key is first needed, the kernel loads it into a free slot. `num_keyslots` in check 3 tells you how many there are.
2. **Tagged requests.** After that, a request carries only two extra numbers: which slot to use, and a *data unit number* worked out from the file and the block's position in it. The UFS driver writes both into the request's transfer descriptor, the small record the controller reads before it moves any data.
3. **Encryption on the fly.** As the controller's DMA pulls the data out of DRAM, the AES engine encrypts it in passing, and the scrambled bytes go out on the wires. On a read it decrypts as the data comes in. The block's number is mixed into the encryption (that's what the "XTS" in AES-256-XTS does), so two identical blocks in different places come out different.

FROM THE CODE The device tree shows the engine as real hardware. It has its own clock selector, `ufs_fde` (`mt6991.dts:6704` and `:6720`), with two sources to pick from: 416 MHz (`ck_univpll_d6`, `:6705`) and 546 MHz (`ck_mainpll_d4`, `:6706`). It has its own clock gate, `ufshci_aes` (`:6714`, `:6730`), and its own reset line, `crypto_rst` (`:6758`; reset bit 2, "ufs-crypto", at `:847`). MediaTek pulses that reset along with the others whenever it resets the controller (`ufs-mediatek.c:276` and `:283`).

SEEN ON THE BOARD During the steady-3000 run, `ck_aes_ufsfde_sel` (the board's name for `ufs_fde`) sat at 416 MHz (main write-up, section 4.2).

1.4 Two more pieces

- **File names** are encrypted too, with a different mode, AES-256-CTS-CBC. The CPU always does that, and the amount of data is tiny, so it never matters for throughput.
- **Metadata encryption.** fscrypt covers only file contents and names. Android can also encrypt everything else on the partition (f2fs's own bookkeeping, file sizes, which blocks are in use) with a second key, through a device-mapper layer called `dm-default-key` that sits under `/data`. It uses the same engine when one is registered, and it skips blocks fscrypt already encrypted, so nothing is encrypted twice.

2 The four checks

Run these as root. `adb root` comes first, because `dmesg` and `dmctl` need it. None of them changes anything on the board.

TRY IT: ALL FOUR CHECKS

```
adb root && adb shell

# 1. Is /data encrypted at all?
getprop ro.crypto.state                # encrypted / unencrypted
getprop ro.crypto.type                 # file = file-based encryption

# 2. What the image asks for, and what is in effect now
grep userdata /vendor/etc/fstab.*
grep ' /data ' /proc/mounts
dmctl list devices                     # only if /data comes from /dev/block/dm-N

# 3. Did the controller hand its AES engine to Linux?
ls /sys/block/sda/queue/crypto/
cat /sys/block/sda/queue/crypto/modes/AES-256-XTS
cat /sys/block/sda/queue/crypto/num_keyslots
dmesg | grep "crypto enable failed"

# 4. Which path did the kernel pick for file contents?
dmesg | grep fscrypt
logcat -b kernel -d | grep fscrypt      # if dmesg has already wrapped
```

Check 1: is `/data` encrypted at all?

Android's init sets these two properties when it mounts `/data` at boot. They answer only the first question, not who does the work.

YOU SEE	MEANING
`encrypted` and `file`	File-based encryption is on. Go on to check 2.
`unencrypted`	No encryption. Stop here: the encryption engine can't be limiting `/data`, and the rest of this document doesn't apply.

Check 2: what the image asks for, and what's in effect

The fstab file is the image's list of mount instructions, one line per partition. `/proc/mounts` is what the kernel is actually doing right now. Look at the `userdata` line in the first and the `/data` line in the second.

WHERE	YOU SEE	MEANING
fstab	`fileencryption=aes-256-xts:aes-256-cts:v2`	The image asks for file-based encryption: AES-256-XTS for contents, AES-256-CTS for names, policy version 2.
fstab	`+inlinecrypt_optimized` after that	A key layout chosen so the engine needs fewer key slots. A sign the image's makers expected a hardware engine.
fstab	`metadata_encryption=…`	Metadata encryption is on too (section 1.4).
mounts	`inlinecrypt` in the options	Path B or C: fscrypt hands file contents to the block layer. Check 3 says which.
mounts	no `inlinecrypt`	Path A: the CPU encrypts inside fscrypt, and the engine sits idle.
mounts	`/data` mounted from `/dev/block/dm-N`	A device-mapper layer sits under `/data`, normally `dm-default-key`. `dmctl list devices` names it, and `dmctl table <name>` should show `default-key`.

FROM THE CODE f2fs adds `,inlinecrypt` to the options it reports if it was mounted with it (`upstream_v6.18/fs/f2fs/super.c:2435`).

Check 3: did the controller hand its engine to Linux?

The folder `/sys/block/sdX/queue/crypto/` exists only if the block layer has an engine registered for that disk. The kernel's own documentation says so: "The presence of this subdirectory … indicates that the device supports inline encryption" (`upstream_v6.18/Documentation/ABI/stable/sysfs-block:242–250`).

FROM THE CODE On the board the folder appears only if all four of these steps succeed:

1. MediaTek's driver declares that the controller has an engine: `hba->caps |= UFSHCD_CAP_CRYPTO` (`ufs-mediatek.c:1757`).
2. At probe, the core reads the controller's crypto capability registers (`ufshcd.c:2350`). It keeps crypto only if the hardware's own capability bit is set as well (`upstream_v6.18/drivers/ufs/core/ufshcd-crypto.c:162`).
3. Every time the controller starts, MediaTek asks its secure firmware to switch the engine on (`ufs-mediatek.c:334–335`, calling `ufs_mtk_crypto_enable()` at `:258`). If the firmware refuses, the driver prints `crypto enable failed` (`:264`) and turns crypto off (`:266`).
4. For each LU, the core registers the engine with that disk's queue (`ufshcd.c:5255`), but only if crypto is still on (`ufshcd-crypto.c:235`).

YOU SEE	MEANING
The folder, with `modes/AES-256-XTS` in it	The engine is registered. The value is a hex bitmask of the block sizes it handles: `0x1000` means 4096 bytes. The kernel's own example, `0x1200`, means 512 and 4096 (`sysfs-block:288–291`).
`num_keyslots`	How many keys the engine can hold at once.

YOU SEE	MEANING
"No such file or directory"	No engine. If check 2 showed `inlinecrypt`, you're on path B: the CPU does the work.
`crypto enable failed` in dmesg	MediaTek's firmware refused. Crypto is off, so there's no folder.

WATCH OUT

- Any `sdX` works. The core registers the same engine on every LU (step 4), so `sda` also answers for the disk that holds `userdata`.
- If there's no `crypto enable failed` line after a long uptime, that proves little: the log may have scrolled past it. The folder is the reliable answer.
- You'll never see `crypto setup failed` (`ufshcd.c:2352`) on a running board. That error stops the UFS probe, so a board that boots from UFS hasn't hit it.

Check 4: which path did the kernel pick?

fscrypt prints one line per encryption mode, once per boot, the first time it sets up a key in that mode. That happens early in boot, when Android unlocks the first keys.

LINE	PATH	MEANING
`fscrypt: AES-256-XTS using blk-crypto (native)`	C	**The controller's engine encrypts file contents.** This is the line that settles it.
`fscrypt: AES-256-XTS using blk-crypto-fallback`	B	The block-layer path was chosen, but the CPU does the work.
`fscrypt: AES-256-XTS using implementation "xts-aes-ce"`	A	The CPU does the work inside fscrypt, with its ARMv8 crypto instructions ("ce" stands for Crypto Extensions).
`fscrypt: AES-256-CTS-CBC using implementation "…"`	none	File names. Always the CPU, tiny amounts of data. Ignore it.
nothing at all	none	Either the line has scrolled out of the log (use the `logcat` line, or grep right after a reboot), or `/data` isn't encrypted (check 1).

FROM THE CODE Paths C and B: `upstream_v6.18/fs/crypto/inline_crypt.c:83` and `:86`. Path A: `fs/crypto/keysetup.c:127`, which prints the driver name `"xts-aes-" MODE` from `arch/arm64/crypto/aes-glue.c:723`, where `MODE` is `"ce"` (`:27`).

NOT YET PROVEN These lines were read in upstream Linux 6.18, not in `android15-6.6`. The sysfs folder from check 3 has been in the kernel since 2022 (its documentation is dated February 2022), well before 6.6, and the fscrypt lines are expected in a 6.6 kernel too. If check 1 says `encrypted` but check 4 prints nothing even right after a reboot, the 6.6 kernel words it differently: look for the matching `pr_info` in its `fs/crypto/`.

Optional: what the kernel was built to support

```
zcat /proc/config.gz | grep -E "BLK_INLINE_ENCRYPTION|FS_ENCRYPTION|SCSI_UFS_CRYPTO|DM_DEFAULT_KEY"
```

OPTION	WHAT IT ALLOWS
`CONFIG_BLK_INLINE_ENCRYPTION`	The block layer can attach a key to each request.
`CONFIG_BLK_INLINE_ENCRYPTION_FALLBACK`	The CPU can stand in when no engine is registered (path B).
`CONFIG_FS_ENCRYPTION_INLINE_CRYPT`	fscrypt may use the block-layer path at all.
`CONFIG_SCSI_UFS_CRYPTO`	The UFS driver can drive the engine.
`CONFIG_DM_DEFAULT_KEY`	The metadata-encryption layer exists. Android-only.

`=y` means built in. A missing line means the feature isn't in this kernel. `/proc/config.gz` exists only if the kernel was built to carry its own configuration.

3 What each answer means for G5

The open question from the main write-up: at G5, writes to `/data` wander between about 3000 and 3700 MB/s, while G4 and G1 stay steady. The leading suspect is the engine's clock sitting at 416 MHz instead of 546. That suspect only exists on path C.

CHECK 4 SAYS	CAN `UFS_FDE` LIMIT `/DATA`?	NEXT STEP
C · `blk-crypto (native)`	Yes. Every byte written to `/data` passes through the engine.	Do the decisive run (main write-up, 4.11): engine clock at 546 MHz during the write, then 10 rounds of G5L2↔G1L2.
B · `blk-crypto-fallback`	No. The engine is idle.	The CPU does AES on every byte, so CPU speed becomes a limit. NOT YET PROVEN That would fit the SQPOLL result (3200 → 3800 MB/s).
A · `implementation "xts-aes-ce"`	No.	As for B, and read the box below: direct I/O isn't direct.
`unencrypted` (check 1)	No.	Look at f2fs garbage collection and discard, WriteBooster, and the CPU.

WATCH OUT: PATH A QUIETLY CHANGES THE BENCHMARK

FROM THE CODE f2fs does direct I/O on an encrypted file only if that file uses the block-layer path. `f2fs_force_buffered_io()` answers "force buffered" when `fscrypt_dio_supported()` says no (`upstream_v6.18/fs/f2fs/file.c:934–939`), and fscrypt says no on path A: "We only support DIO with inline crypto, not fs-layer crypto" (`fs/crypto/inline_crypt.c:453`). `f2fs_should_use_dio()` then returns false (`file.c:4723`, `:4731`), and the write goes through the normal buffered path with no error.

So on path A, fio's `direct=1` never reaches the disk directly. Writes land in RAM first, in the page cache, and the kernel flushes them to the disk in bursts. What fio measures then depends on how much unwritten data has piled up in RAM, which is exactly the kind of thing that makes numbers wander from run to run.

NOT YET PROVEN That this happens on the board. Checks 2 and 4 tell you whether you're on path A at all.

TRY IT: A READ-ONLY LOOK AT THE ENGINE

A read of the raw disk carries no key, so it never goes through the engine. A read of a big file in `/data` does, on path C. If both are equally fast and steady at G5, the engine isn't what limits reads. Writes could still differ, but this test is safe to run any time.

```
ls -l /dev/block/by-name/userdata       # for example -> /dev/block/sdc39: the disk is sdc
echo 3 > /proc/sys/vm/drop_caches        # empty the page cache first

fio --name=raw  --filename=/dev/block/sdc --readonly --rw=read --bs=512k \
    --iodepth=64 --ioengine=io_uring --direct=1 --size=4g --runtime=20 --time_based

fio --name=file --filename=/data/local/tmp/fio_4g.bin --readonly --rw=read --bs=512k \
    --iodepth=64 --ioengine=io_uring --direct=1 --size=4g --runtime=20 --time_based
```

`--readonly` makes fio refuse to write, which matters on a raw disk. For the second run, use a file the write runs already created; the name here is only an example. The comparison only means something on path C. On path A the file read goes through the page cache (the box above), and its numbers aren't comparable.

4 Reference: changes, sources, glossary

4.1 What this changes in the main write-up

- Section 4.5 says the `inlinecrypt` mount option "chooses the hardware". More exactly, it chooses the block-layer path. The hardware does the work only if the engine is registered (check 3). Otherwise the CPU does it through `blk-crypto-fallback` (path B).
- Section 4.5's "Try it" box had two of these checks. This document adds the properties, the sysfs folder, the fscrypt line that names the path, and the f2fs direct-I/O rule.

4.2 Where each claim comes from

CLAIM	FILE AND LINE, IN `SOURCES/`
MediaTek declares the engine	`mtk_vendor_6.6/ufs-mediatek.c:1757`
Secure-firmware switch-on, and what a refusal does	`ufs-mediatek.c:258–267`, called at `:334–335`
Crypto reset pulsed with the others	`ufs-mediatek.c:276`, `:283`
Engine clock, its sources, gate and reset	`mtk_vendor_6.6/mt6991.dts:6704–6706`, `:6714`, `:6720–6722`, `:6730`, `:6758`, `:847`
Core reads crypto capabilities at probe	`android15-6.6/ufshcd.c:2350`, failure message `:2352`
Engine registered on each LU	`android15-6.6/ufshcd.c:5255`
Hardware bit required; registration only while crypto is on	`upstream_v6.18/drivers/ufs/core/ufshcd-crypto.c:162`, `:235`
What the sysfs folder means, and its value format	`upstream_v6.18/Documentation/ABI/stable/sysfs-block:242–299`
The sysfs files themselves	`upstream_v6.18/block/blk-crypto-sysfs.c:41–50`
fscrypt's log lines	`upstream_v6.18/fs/crypto/inline_crypt.c:83`, `:86`; `fs/crypto/keysetup.c:127`
arm64 AES driver name	`upstream_v6.18/arch/arm64/crypto/aes-glue.c:27`, `:723`
f2fs reports `inlinecrypt`	`upstream_v6.18/fs/f2fs/super.c:2435`
f2fs refuses direct I/O on path A	`upstream_v6.18/fs/f2fs/file.c:934–939`, `:4723`, `:4731`; `fs/crypto/inline_crypt.c:444–453`

The upstream files come from the shallow Linux 6.18 clone in `~/Personal_Study/kernel/linux` (5 December 2025), copied with their kernel paths. Android properties, fstab syntax, `dmctl` and `logcat` are Android behaviour and aren't in these copies.

4.3 Glossary

AES-256-XTS

The cipher mode for file contents. XTS mixes each block's position into the encryption, so identical blocks encrypt differently.

AES-256-CTS-CBC

The cipher mode for file names. Always done by the CPU.

blk-crypto

The block layer's support for attaching a key to each request, so hardware further down can do the encryption.

blk-crypto-fallback

blk-crypto's CPU stand-in for disks without an engine (path B).

Bounce page

A spare page that holds the encrypted copy of a block while it's being written, so the page cache keeps the plain version. Paths A and B use them; path C doesn't need them.

Data unit number (DUN)

The per-block number sent with each request and mixed into the encryption.

dm-default-key

Android's device-mapper layer for metadata encryption.

FBE, file-based encryption

Android's scheme in which files get their own keys, applied per file by fscrypt.

fscrypt

The kernel's file-encryption layer, used by f2fs and ext4.

fstab

The image's list of mount instructions, one line per partition.

Inline encryption

Encryption done by hardware on the data path itself. Here, the AES engine in the UFS controller (path C).

Key slot

A register in the engine that holds one key.

`ufs_fde`, `ck_aes_ufsfde_sel`

The engine's clock selector: its device-tree name, and its name in the board's `clk_summary`.

`ufshci_aes`

The engine's clock gate.

UFSHCI

The UFS host controller standard. Its crypto section defines the key slots and the tagged requests.

vold

Android's storage daemon. It creates and unlocks the encryption keys.

Written with Claude Code on 1 November 2026, from the copies in `sources/`. Every command in this document only reads.